

CS Lab 1 - Client-Side Message Encryption

Repo: https://github.com/SMaityCodes/CS_Lab1

Objective: Explore the existing app in the repo above (a small Node.js/Express app with a SQL backend — the README explains it fully), then modify its personal-message feature so the message is encrypted in the browser before it's saved. First hands-on use of hashing and AES via the Web Crypto API.

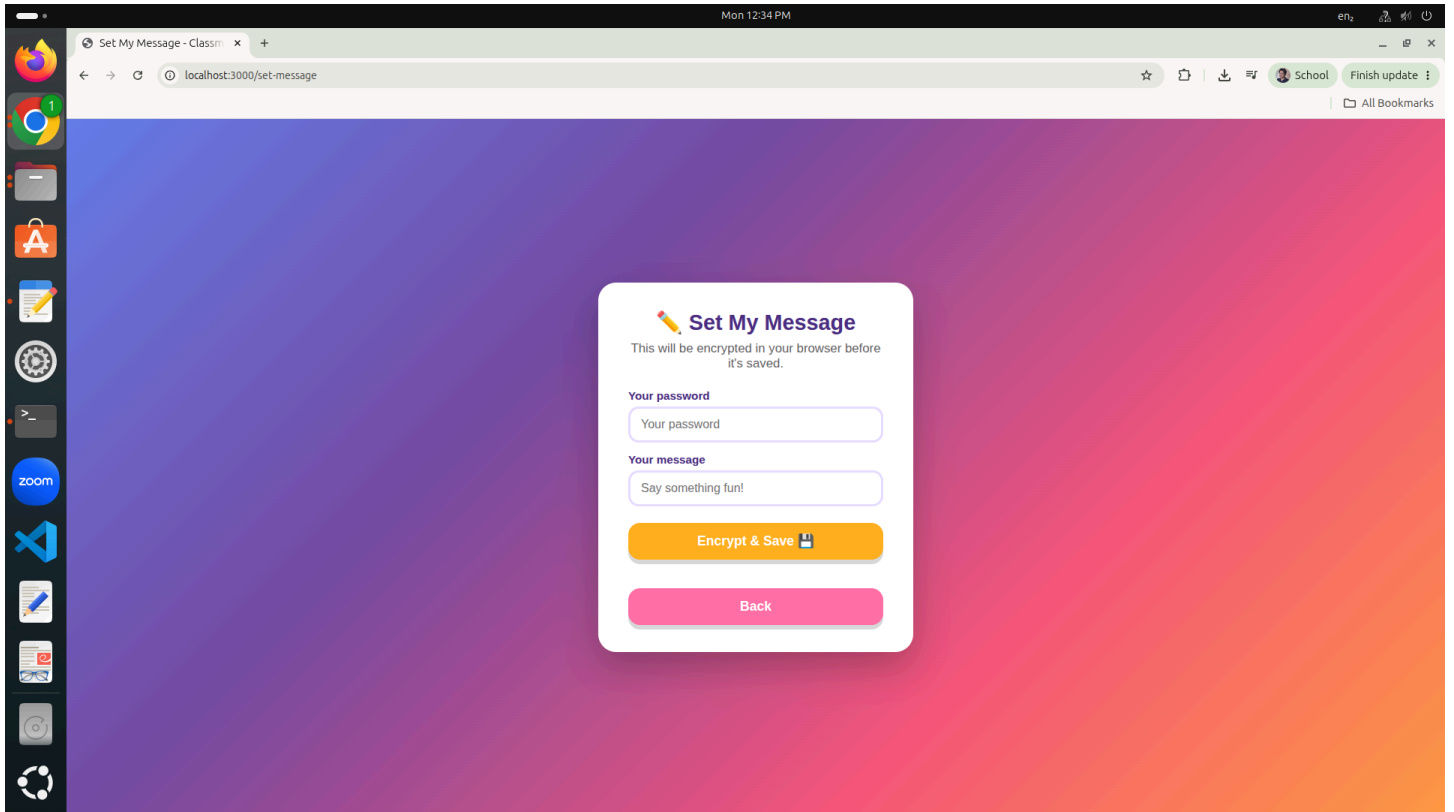
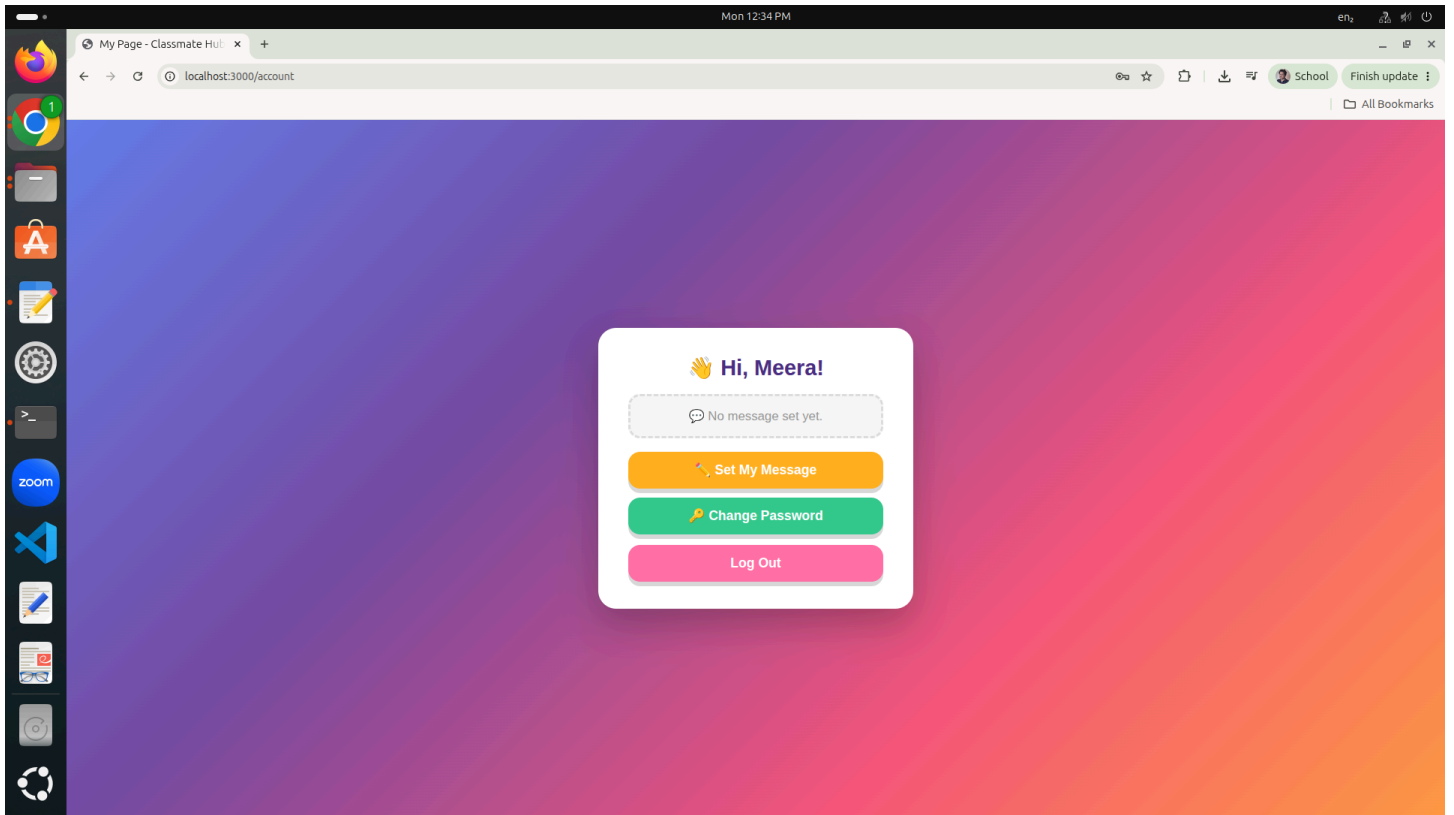
Part A — Explore First (do this before writing any code)

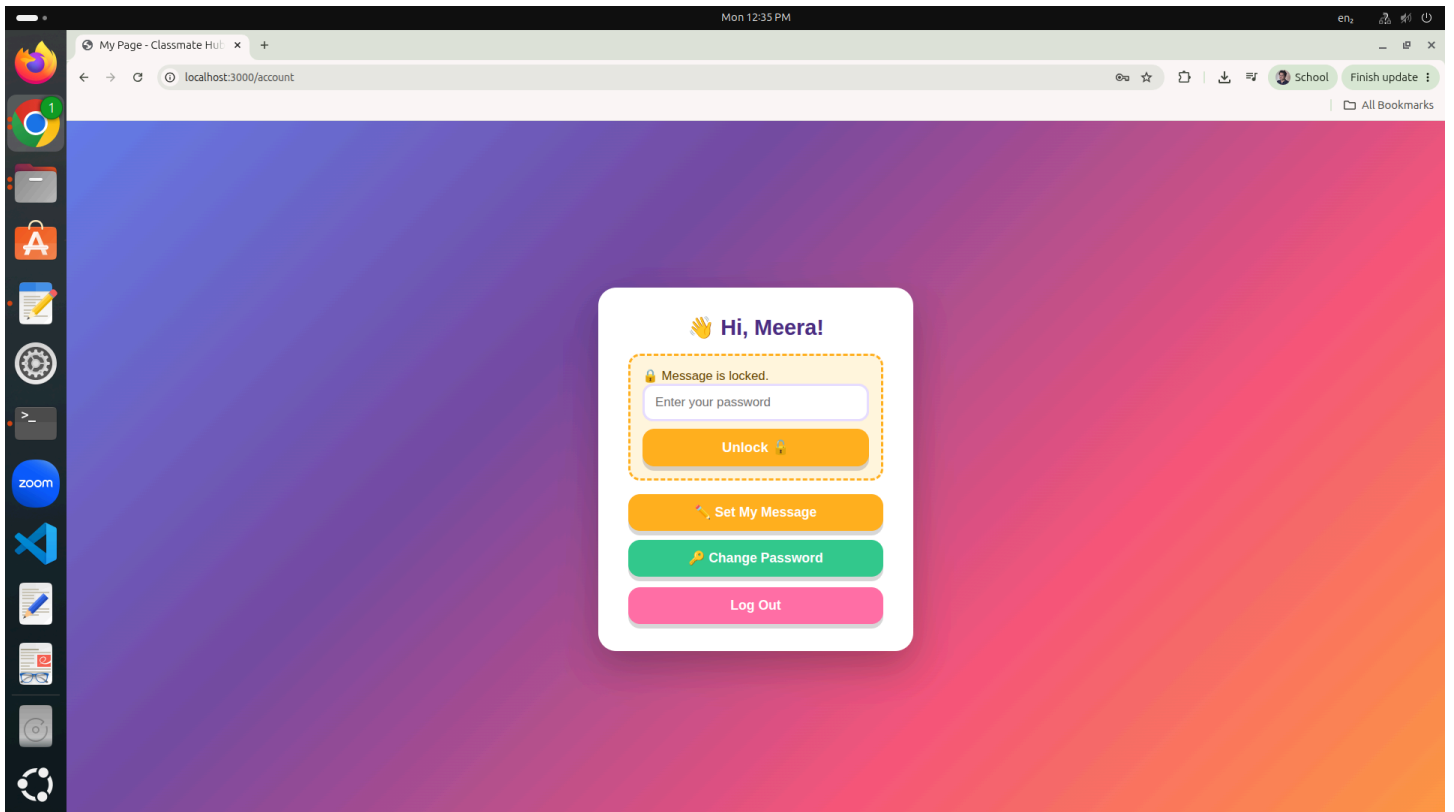
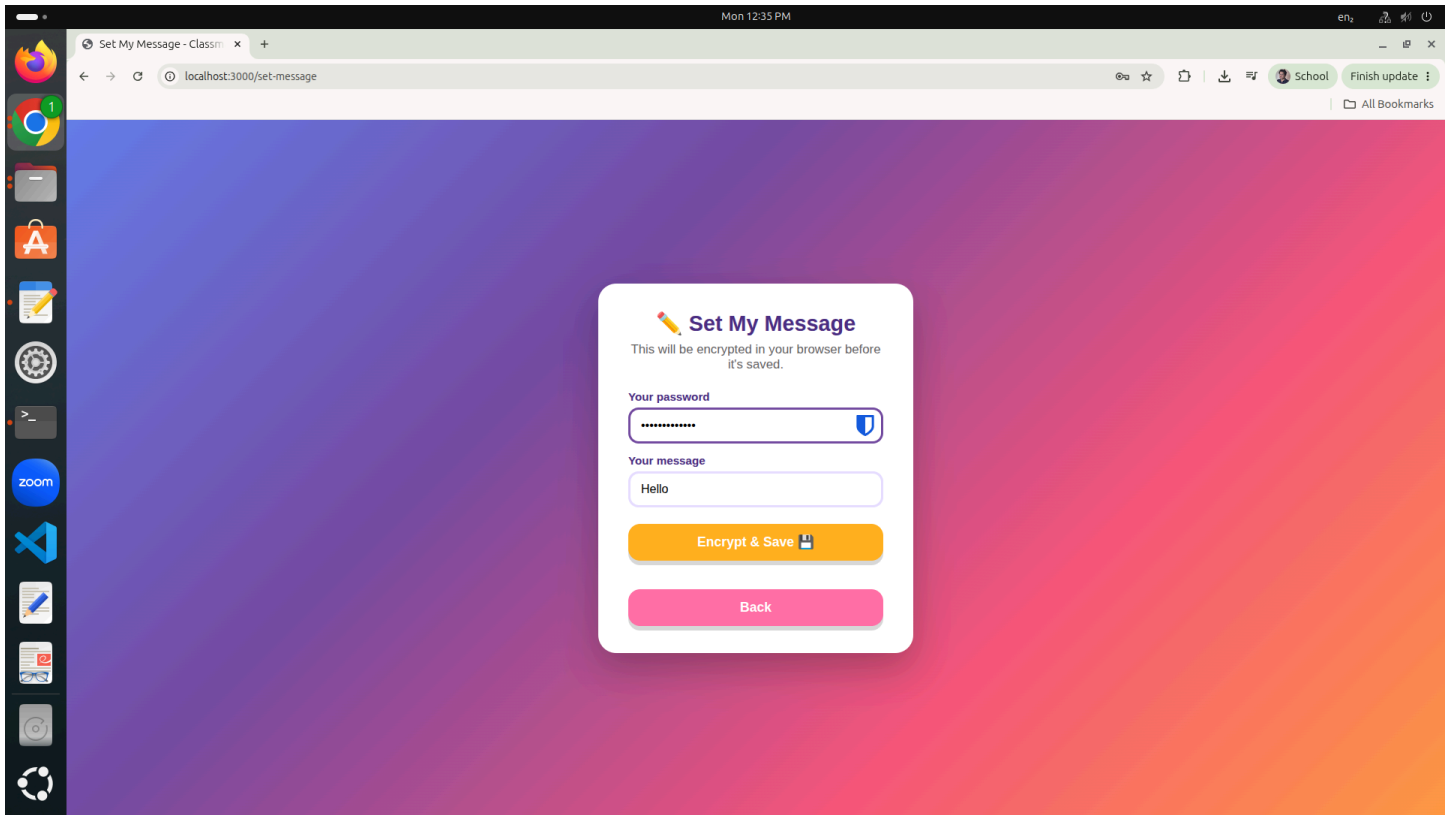
Clone, install, and run the app; log in and use every page. Then read the code and be ready to explain: the accounts table schema; how “Set My Message” works end-to-end today; how the account page currently fetches/displays the message; how the server knows who is logged in (cookies).

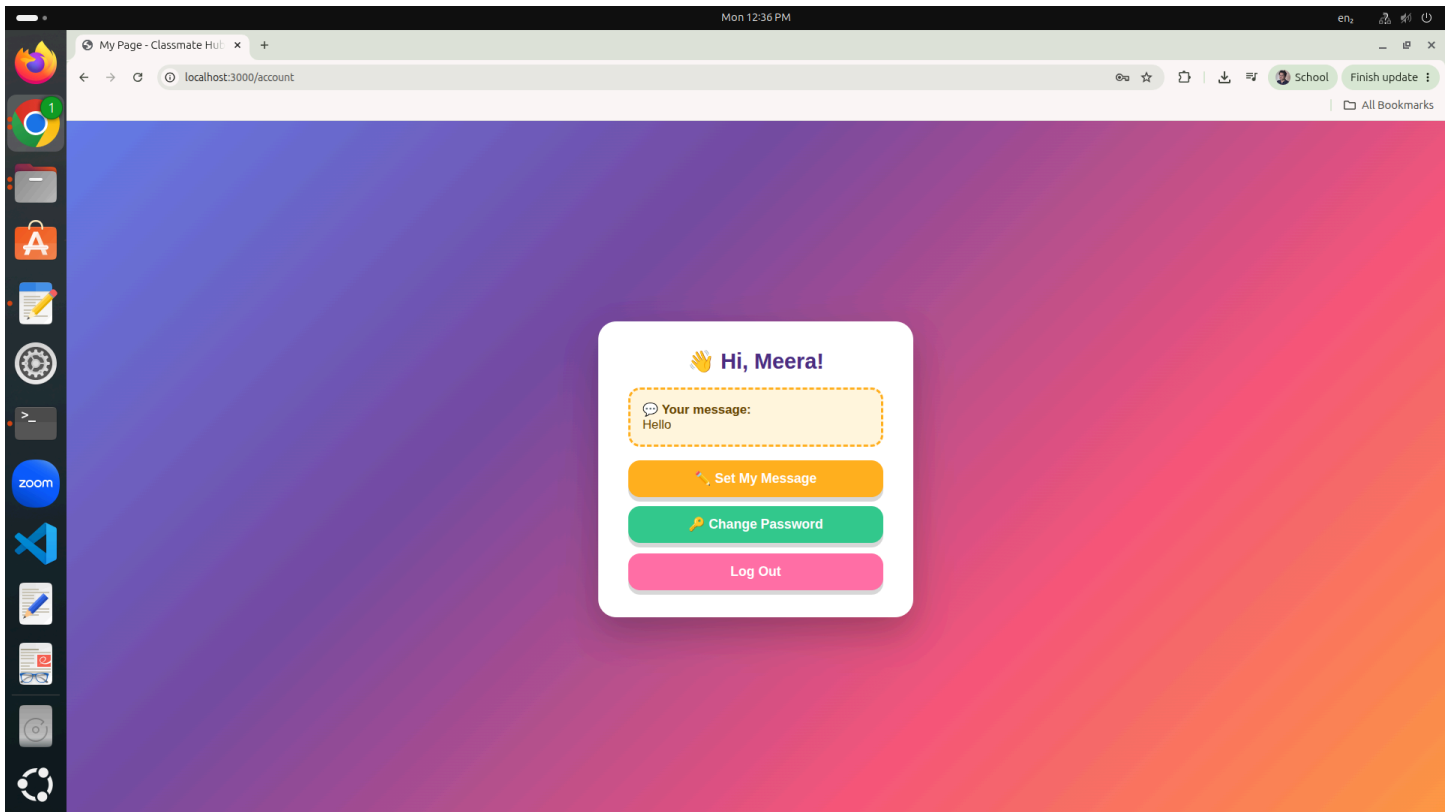
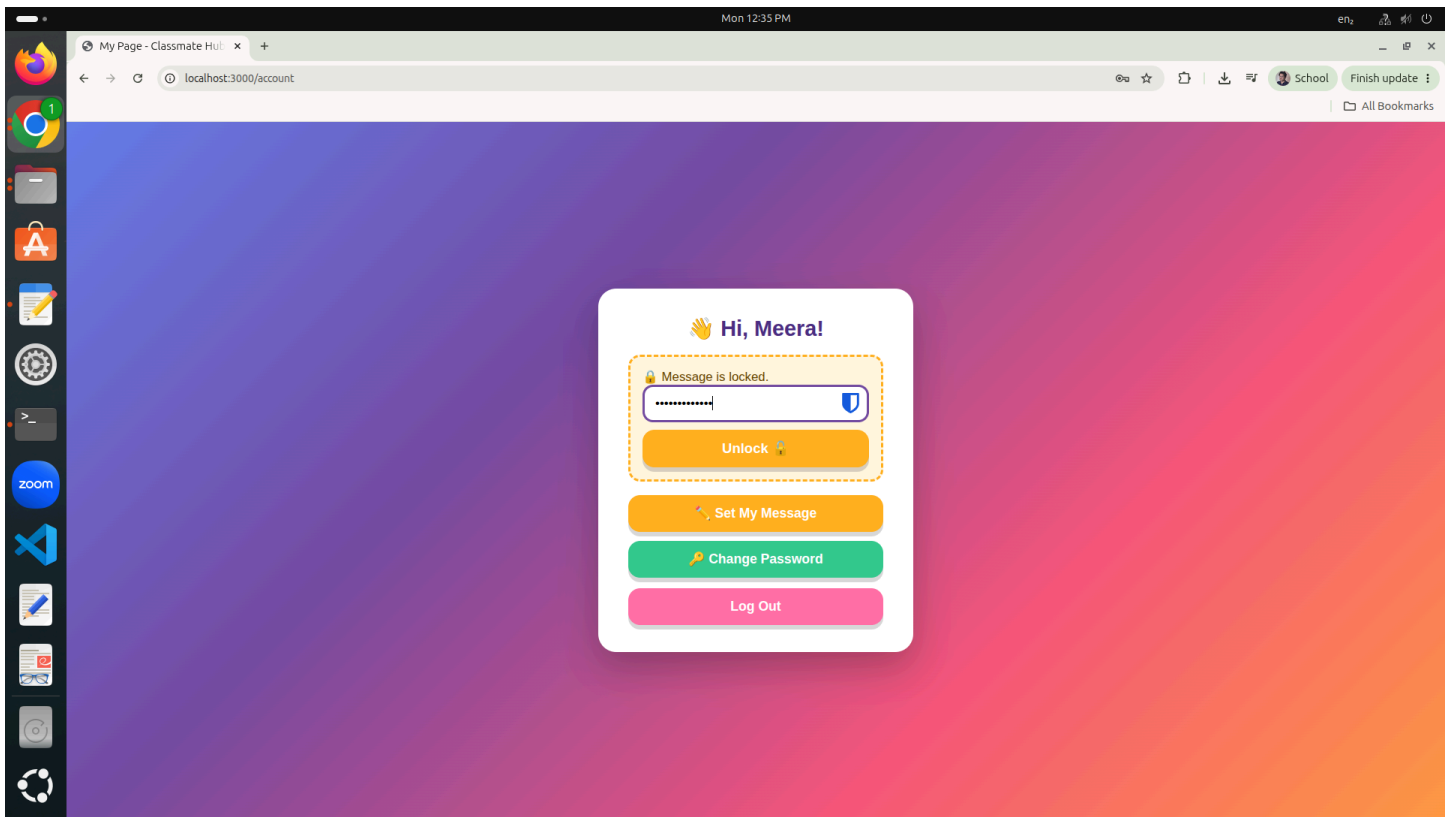
Part B — Modify the Message Feature

1. Hash the user's password with the Web Crypto API (`crypto.subtle.digest`, SHA-256 or MD5). Use the hash directly as the AES key — no salting/PBKDF2 needed for this assignment.
2. Encrypt the message client-side with AES-GCM (`crypto.subtle.encrypt`) before it is ever submitted. Generate a fresh random IV each time (`crypto.getRandomValues`) and store it alongside the ciphertext (e.g., a new DB column).
3. Only ciphertext + IV may ever reach the server or the database.
4. On the account page, do not show the message immediately — show a locked state with a password box + Unlock button. Decryption must happen entirely client-side with zero network request, using ciphertext/IV already present on the page.

Sample screenshots of what is expected:-







Scope — read carefully (this is not a build-from-scratch task)

- You will likely modify: the schema file, the message-page route(s), and the account-page route. **Add exactly one new client-side file, placed at `public/crypto.js`, containing all your hashing/encryption/decryption functions.**
- `crypto.js` must be a thin wrapper around the browser's own built-in `crypto.subtle` API — it does not implement any cryptography itself, it just gives cleaner function names to calls like `digest()`, `importKey()`, `encrypt()`, and `decrypt()`. Keeping every cryptographic operation centralized in this one file, inside `/public`, is a strict requirement, not a suggestion — do not split this logic across multiple files or duplicate it elsewhere.**

- You must NOT touch: login, change-password, the server entry file, or the shared page-template file. If you do, reconsider your approach.
- No front-end frameworks, bundlers, or external crypto libraries — only the browser's built-in Web Crypto API.

Submit

Your Github repo link with a readme explaining the installation and execution guidelines. Include screenshots.

Note: you may use any resources or tools, but you must be able to explain every part of your own submitted code in your own words — submissions the author cannot explain receive no credit, regardless of whether the code runs.

Explain to the Evaluators:-

- First run your app to confirm it behaves correctly - show the locked account page; the unlocked/decrypted account page
- A brief explanation of your approach.
- Show the database showing ciphertext stored (not readable text);
- Show the DevTools Network tab on Save (showing only ciphertext/IV, not plaintext or password);
- Show the Network tab during Unlock (showing no request was made).
- You'll be asked to explain your public/crypto.js file directly (what each function does and why), plus the one line in your code that calls it before a message is submitted. Be ready to walk through your own file live.

HAPPY LEARNING

